Rapid Tech Insights

ENGINEERING REPORT
Rapid Inventory · Fixes, verification and release readiness

Solutions — Rapid Inventory

Rapid Tech Insights · One-page summary · 29 October 2026

Five fixes, each in a few lines. The full story is in ROOT_CAUSE_ANALYSIS.md, and the raw evidence is in `release/`.

---

1. Overselling

- **Problem:** two orders could read the same stock at the same moment, and each overwrote the other. 40 orders were accepted against 10 units in stock.
- **Fix:** check and deduct stock in one database statement (`… WHERE Stock >= @qty`), inside one transaction with the order. A database rule blocks negative stock.
- **Proof:**
  - Last-unit race across two API instances: one success, one refusal.
  - 40 orders against 10 in stock: exactly 10 accepted.
  - Two live API processes: 41 + 39 = 80 sold from 80.
- **Where:** `backend/Services/OrderService.cs` · `tests/Inventory.Api.Tests/OrderConcurrencyTests.cs`

2. Slow inventory page

- **Problem:** one extra database query per product to fetch its category: 202 queries per page.
- **Fix:** one query with a join, so 2 queries per page. No index needed.
- **Proof:** a typical page load went from 225 ms to 12 ms, on the same data, returning the same response.
- **Where:** `backend/Controllers/ProductsController.cs` · `tests/Inventory.Api.Tests/ProductListTests.cs`

3. Live board not updating for others

- **Problem:** stock updates went only to the person who placed the order.
- **Fix:** send updates to everyone watching; keep reconnecting if the connection drops.
- **Proof:** in two real browser sessions, the second updated without a refresh; failed orders sent nothing.
- **Where:** `backend/Services/IStockNotifier.cs` · `frontend/src/app/stock-realtime.service.ts` · `seed/two-browser-check.mjs`

4. Order logic in the controller

- **Problem:** the controller did everything, and an unused copy of the logic had drifted out of date. Cancels also had a race: 6 of 10 at once each returned the stock.
- **Fix:** `OrderService` does the work; the controller only handles HTTP. Cancels are now safe.
- **Proof:** 13 API behaviour tests pass identically before and after; the cancel race test passes.

- **Where:** `backend/Services/OrderService.cs` · `backend/Controllers/OrdersController.cs` · `tests/Inventory.Api.Tests/OrderContractTests.cs`

5. Low-stock alerts

- **Problem:** alerts were sent but only shown on the Live board page.
- **Fix:** a dismissable toast on every page, for every user. It fires only when stock drops **below** the reorder level.
- **Proof:**
  - 8 rule tests and 17 front-end tests pass.
  - Live: nothing at exactly 15; at 14 the alert appeared in both browsers.
- **Where:** `frontend/src/app/low-stock-alerts.ts` · `tests/Inventory.Api.Tests/LowStockAlertTests.cs`

Overall: 42/42 backend tests and 17/17 front-end tests pass, and both builds are clean.

Still open:

- Live updates need a backplane before running more than one API server.
- There is no duplicate-order protection for client retries.
- The walkthrough video is not recorded yet.